

习题六实验报告

2313725 张耕嘉

一、实验目的

本实验旨在通过构造字符串的哈希值，探索不同哈希构造方法的特性，并比较其性能表现。实验中使用两种不同的方法对字符串进行哈希值计算，分别是基于字母的多项式哈希方法和基于音节块的哈希方法。通过对两篇英文文章的处理，统计哈希表中单词查找的平均查找长度，从而分析不同哈希方法的优缺点。

二、实验原理

哈希函数是一种广泛应用于计算机科学领域的技术，用于将输入数据映射到固定长度的哈希值。理想的哈希函数具有快速计算、低冲突率和均匀分布的特点。本实验基于以下两种方法构造字符串哈希值：

1. 基于字母的多项式哈希

- 基本原理：使用多项式模型计算单词的哈希值，将每个字符的 ASCII 值乘以其对应的权重后累加。示例："abcd" 的哈希值为 $a*g^3 + b*g^2 + c*g^1 + d*g^0$ 。
- 关键参数：
 - 哈希基数选择为 31，这是一个常用的素数，用于提高哈希值分布的均匀性。
 - 取模操作使用大素数 MOD (如 10^9+7)，用于限制哈希值的大小并减少溢出。
- 实现特点：
 - 对字符的顺序敏感：不同排列的字符串（如 "abcd" 和 "dcba"）产生不同的哈希值。
 - 易受高频单词冲突影响：常见单词（如 "the"）容易出现冲突。

2. 基于音节块的哈希

- 基本原理：使用正则表达式提取单词中的音节块（如元音和辅音的组合），并对每个音节块计算哈希值后累积。示例："beautiful" 提取音节块为 "beau", "ti", "ful"，并累积各音节的哈希值。
- 音节提取方法：
 - 使用正则表达式匹配模式 $([aeiou]^*[aeiou]^+)$ ，提取以一个或多个辅音开头并跟随元音的音节块。
- 关键参数：
 - 每个音节块按照类似多项式哈希的方法进行编码。
 - 权重和取模操作与多项式哈希类似。
- 实现特点：
 - 对音节块的顺序敏感：音节顺序的改变会导致不同的哈希值。
 - 冲突率低：由于音节的分布特性，更适合复杂单词的区分。

三、实验步骤

1. 文章选取

- 第一篇文章：用于构建哈希表：
The quick brown fox jumps over the lazy dog. The quick fox is clever and agile.
- 第二篇文章：用于查找单词并统计平均查找长度：

A quick brown fox once jumped over a lazy dog. The dog barked at the fox, but the fox ignored it and ran away.

2. 代码实现

(1) 多项式哈希伪代码

```
function polynomial_hash(word):  
    MOD = 10^9 + 7  
    BASE = 31  
    hash_value = 0  
    power = 1  
    for each char in word:  
        hash_value = (hash_value + (ASCII(char) * power) % MOD) % MOD  
        power = (power * BASE) % MOD  
    return hash_value
```

(2) 音节块哈希伪代码

```
function syllable_hash(word):  
    MOD = 10^9 + 7  
    BASE = 31  
    hash_value = 0  
    power = 1  
    syllables = extract_syllables(word) // 使用正则表达式提取音节块  
    for each syllable in syllables:  
        for each char in syllable:  
            hash_value = (hash_value + (ASCII(char) * power) % MOD) % MOD  
            power = (power * BASE) % MOD  
    return hash_value
```

3. 哈希表构造与查询

插入伪代码

```
function insert_to_hash_table(hash_table, word, hash_function):  
    hash_value = hash_function(word)  
    index = hash_value % size_of(hash_table)  
    while hash_table[index] is not empty:  
        index = (index + 1) % size_of(hash_table) // 线性探测  
    hash_table[index] = word
```

查询伪代码

```
function find_in_hash_table(hash_table, word, hash_function):  
    hash_value = hash_function(word)  
    index = hash_value % size_of(hash_table)  
    probes = 0  
    while hash_table[index] is not empty:  
        probes += 1  
        if hash_table[index] == word:  
            return probes  
        index = (index + 1) % size_of(hash_table)  
    return probes
```

4. 性能指标

- 对比基于字母和基于音节块的平均查找长度，评估冲突率和分布均匀性。

四、 实验结果

运行程序后，得到以下结果：

- 基于字母的平均查找长度： 1.83333
- 基于音节块的平均查找长度： 1.33333

五、 实验分析

1. 基于字母的多项式哈希

- 平均查找长度较高 (1.83333)。
- 原因分析：
 - 直接基于字母编码的哈希方法对字符顺序敏感。
 - 对于频率较高的单词（如 "the"）容易发生冲突。

2. 基于音节块的哈希

- 平均查找长度较低 (1.33333)。
- 原因分析：
 - 音节块哈希考虑了单词的音节结构，编码冲突更少，分布更均匀。
 - 复杂单词（如 "beautiful"）在此方法下具有更高区分度。

3. 两种方法比较

- 音节块哈希在处理复杂单词时更高效，且更适合应用于语义相关的文本数据。
- 多项式哈希更适合快速处理简单字符串。

六、 结论

- 基于音节块的哈希方法在冲突率和平均查找长度方面优于基于字母的多项式哈希。
- 哈希基数 (31) 和取模运算有效减少了整数溢出，同时保证了哈希值的均匀性。
- 在自然语言处理任务中，优先选择音节块哈希以降低冲突率。
- 对于频率较高的常见单词，可以结合停用词表优化哈希表性能。